

Modeling and Verifying the Ariadne Protocol Using CSP

Xi Wu¹, Si Liu¹, Huibiao Zhu¹, Yongxin Zhao¹ and Lei Chen²

¹Shanghai Key Laboratory of Trustworthy Computing, East China Normal University

²Department of Computer Science and Technology, East China Normal University

Email: {xiwu, siliu, hbzhu, yxzhao}@sei.ecnu.edu.cn, lchen@cs.ecnu.edu.cn

Abstract—Mobile Ad Hoc Networks (MANETs) are formed dynamically by mobile nodes without the support of prior stationary infrastructures. In such networks, routing protocols, particularly secure ones are always an essential part. Ariadne, an efficient and well-known on-demand secure protocol of MANETs, mainly concerns about how to prevent a malicious node from compromising the route. In this paper, we apply the method of process algebra Communicating Sequential Processes (CSP) to model and reason about the Ariadne protocol, focusing on the process of its route discovery. In our framework, we consider the communication entities as processes, including the initiator, the intermediate nodes and the target. Moreover, we use PAT, a model checker for CSP, to verify whether the model caters for the specification and the non-trivial secure properties, e.g. existence of fake path. Our verification result naturally demonstrates that the fake routing attacks may be present in the Ariadne protocol.

Keywords—Formal Verification; CSP; Mobile Ad hoc Networks; Ariadne;

I. INTRODUCTION

Wireless communication technology has become one of the most promising technologies. Mobile Ad Hoc Networks (MANETs) consist of groups of wireless mobile devices (laptops, PDAs, sensors, etc.), being completely self-configuring and self-organizing, and are independent of any existing fixed infrastructure. Moreover, nodes can move arbitrarily within, join in, or leave for the network dynamically, which makes the whole network quickly and easily set up as needed. Due to these novel features, MANETs have been widely applied in many fields including military, ambient intelligence and emergency contingencies.

In such networks, routing protocols ([5], [6], [16], [19]), particularly secure ones are always an essential part since it is a major concern about how to prevent a malicious node from compromising the route. Malicious nodes may cause some typical security issues such as the attacks of denial-of-service and tunneling which redirect the traffic of the networks, the attacks of spoofing that the intruder node may masquerade as the other nodes, and the attack called fabrication of false routing messages. Ariadne [6], as an extension to the dynamic source routing (DSR) protocol [7], proposed by Hu *et al.*, is a new secure on-demand ad hoc network routing protocol for preventing attackers and security vulnerabilities.

Many research efforts have been addressed to analyze and improve the Ariadne protocol. Hu *et al.* evaluated its performance based on simulation [6]. Sivakumar *et al.* proposed some modifications to improve its resiliency [17]. All these works, however, do not investigate the protocol using formal methods and may not take into account the security and correctness. In addition, Lin *et al.* have already found some drawbacks of this protocol [8] and Buttyán *et al.* applied a mathematical framework in analyzing the protocol and finding out attacks on it ([1], [3]). They have done well in analyzing the protocol, if only they had given some verifications. In formal literature, as far as we know, only Pura *et al.* have already modeled the Ariadne protocol using HLPSL and applied AVISPA to validate its security properties [14]. They focus more on the use of the tools than the analysis of the protocol itself. Thus, the research for the Ariadne protocol is still challenging. In this paper, we use formal methods to model the protocol and use the tool to verify whether the model caters for the specification and the non-trivial secure properties.

Lowe *et al.* first apply the method of process algebra Communicating Sequential Processes (CSP) to model and analyze a security protocol, the TMN protocol [10]. CSP is a well-known process algebra in modeling and verifying the reliability and the security in concurrent systems and widely used in ([9], [10], [11], [15]). Moreover, a lot of automated model checkers for analyzing and understanding systems described by CSP have been produced, such as Process Analysis Toolkit (PAT) [18]. Inspired by Lowe's work, we use CSP to model the Ariadne secure routing protocol. This protocol has two phases: the route discovery and the route maintain. Due to the facts that the route maintain is based on the route discovery and the intrusion tends to occur in the discovery phase, in this paper, we focus on the Ariadne route discovery. We abstract the protocol, that the initiator, the intermediate nodes and the target are described as processes and all of these communication entities share the global clock. It achieves the effect of asymmetric key encryption through clock synchronization and time delay. Besides, we also propose an intruder model in which the intruder can eavesdrop, fake, intercept and deduce the message. Furthermore, by exploring PAT [18], we verify the security properties of the Ariadne protocol

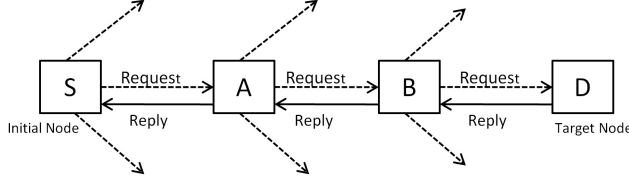


Figure 1. An example of the route discovery in Ariadne

model and we find that the fake routing attacks may be present in the protocol, which have been pointed out in ([1],

[3]). The rest of this paper is organized as follows. We give an overview of the Ariadne secure routing protocol in Section 2 and in this section, we also introduce the CSP method in details. Section 3 is about the formalizing of the protocol. We abstract the communication entities as processes, propose an intruder model and produce a CSP description of the specifications. In Section 4, based on the analysis of traces, we use model checker PAT to implement and verify the achieved model with four properties such as Deadlock Freedom, Message Consistency, Node List Security and Fake Path Existence. In section 5, we conclude the paper and present the future directions.

II. OVERVIEW OF THE ARIADNE PROTOCOL

Ariadne, as an extension to the dynamic source routing (DSR) protocol, is composed of routing discovery and routing maintain. One of its main security goals is to prevent attackers or compromised nodes from tampering with uncompromised routes consisting of uncompromised nodes, and also prevent many types of Denial-of-Service attacks. And it also provides a property that no intermediate node can remove a previous node in the node list in the request or reply, which means it can prevent a compromised node from removing a node from the node list arbitrarily

[6]. To achieve the security goal as mentioned, the Ariadne protocol applies three authentication mechanisms: TESLA protocol ([12], [13]), which is used for route data authentication to certificate the integrity and authenticity of the routing message; End-to-End nodes authentication mechanism, which is used to verify the authenticity and freshness of the request and reply using the shared key; Per-hop hashing authentication mechanism, which is used to prevent an attacker from removing a node from the node list.

A. Notations

Now, we introduce the following notations before describing the protocol: S stands for the initiator and D for the target; A and B are participants, such as intermediate nodes; C denotes an internal node captured by external intruders; $*$ stands for all nodes in the network; K_{SD} and K_{DS} ,

used to certificate the identities of the communicating nodes, represent the secret MAC keys shared between S and D . $MAC_{K_{SD}}(M)$ denotes the MAC value calculated by K_{SD} and message M . Using the notations as mentioned, we next describe the process of Ariadne routing discovery and its authentication mechanism.

B. Description of the Ariadne Routing Discovery and Authentication Mechanism

When there is a packet needed to be sent to the destination, the initiator node first checks its local table. If there is already some path to the target, the initiator will send the packet along the existing path; Otherwise, it will start the route discovery. The process of the Ariadne protocol route discovery is divided into two parts: the initial node broadcasts a routing request to all of its neighbours and the target node sends a routing reply back to the initial node after it gets the request. In order to illustrate the process clearly, we give a simple topology example in Figure 1, where the dotted line means broadcast and the solid line stands for unicast.

Broadcasting the route request According to Figure 1, S broadcasts a route request to all of its neighbors in the whole network. In Ariadne, a route request contains four fields such as $\langle msg_{req}, hash, node\ list, MAC\ list \rangle$. Here, msg_{req} stands for the route request message and $hash$ value is calculated by a per-hop hash function $H(x)$ except the first element which is computed as $MAC_{K_{SD}}(msg_{req})$. Through the per-hop hash function $H(x)$, Ariadne protocol ensures that a malicious node cannot remove the intermediate node or modify the order of the node list arbitrarily. $node\ list$ and $MAC\ list$ store the addresses and the MAC values respectively, which are empty initially. msg_{req} also has five fields, $\langle request, initiator, target, id, time\ interval \rangle$. Here we ignore the details of the msg_{req} for simplicity. The process of the broadcasting is shown in Table 1.

Table 1. Broadcasting the route request

$S :$	$msg_{req} = (request, S, D, id_{SD}, t_i)$
	$h_S = MAC_{K_{SD}}(msg_{req})$
$S \rightarrow * :$	$\langle msg_{req}, h_S, (), () \rangle$
$A :$	$h_A = H[A, h_S]$
	$M_A = MAC_{K_{A t_i}}(msg_{req}, h_A, (A), ())$
$A \rightarrow * :$	$\langle msg_{req}, h_A, (A), (M_A) \rangle$
$B :$	$h_B = H[B, h_A]$
	$M_B = MAC_{K_{B t_i}}(msg_{req}, h_B, (A, B), (M_A))$
$B \rightarrow * :$	$\langle msg_{req}, h_B, (A, B), (M_A, M_B) \rangle$

When the intermediate nodes receive the route request, they will first verify the authenticity of the route request message and the effectiveness of the key that the previous node uses. If the TESLA key has already been released, the node will discard this message and send an error message

back to the initial node. Otherwise, the current node adds its address, hash value and the MAC value computed by its own TESLA key into the request message and rebroadcasts it.

Unicasting the route reply After D receives the route request message, it checks the consistency of the *node list* and also tests whether any key the nodes use has already been released within the specified time or not. Only in the case that all the conditions we have mentioned above are satisfied will D accept the route request and send a route reply back to S along the reverse order of the nodes in the *node list*.

In Ariadne, a route reply contains three fields, $\langle msg_{rep}, target\ MAC, key\ list \rangle$. Here, *target MAC* stands for the MAC value of the target node and *key list* is empty initially. Besides, msg_{rep} consists of six fields, $\langle reply, target, initiator, time\ interval, node\ list, MAC\ list \rangle$. Table 2 illustrates the process of unicasting the route reply.

Table 2. Unicasting the route reply

$D :$	$msg_{rep} = (reply, D, S, t_i, (A, B), (M_A, M_B))$ $M_D = MAC_{k_{DS}}(msg_{rep})$
$D \rightarrow B :$	$\langle msg_{rep}, M_D, () \rangle$
$B \rightarrow A :$	$\langle msg_{rep}, M_D, (K_{B_{t_i}}) \rangle$
$A \rightarrow S :$	$\langle msg_{rep}, M_D, (K_{B_{t_i}}, K_{A_{t_i}}) \rangle$

When an intermediate node receives the route reply, it caches the message until its TESLA key is released. Then the node adds its own key into the *key list* and sends the message to the last node. When S receives the reply, it will check the correctness of each key and each MAC value respectively. Besides, it will also verify the MAC value of D . Only when there is no error in the process of the validation would S accept the route reply and cache the path in its local table. Otherwise, S will discard the reply.

C. Assumptions

We naturally inherit all of the assumptions of the Ariadne protocol and the TESLA protocol. In order to facilitate the modeling in the next section, we also list the following assumptions:

- 1) There is no efficient routing path between S and D in the local table of S .
- 2) Network is bidirectional [6], i.e. if A can receive the message from B , then B must be able to receive the message from A .
- 3) Each intermediate node has a TESLA one-way key chain and the keys in the chain are computed through the function $F(x)$. Each node firstly releases one key to all nodes in its broadcast range so that the follow-up nodes can use it to certificate the message. More

details about TESLA protocol can be found in ([12], [13]).

- 4) C , captured by intruders, can get initial information of all the nodes participating the process of route discovery. It can intercept or eavesdrop or fake messages passed between nodes and it can also be used as a normal node to join in the routing process.

D. A Brief Introduction to CSP Method

Here we give a brief introduction to CSP. More details about CSP can be found in ([2], [4]). CSP processes are composed of primitive processes and actions. P and Q represent processes which have alphabets $\alpha(P)$ and $\alpha(Q)$ to denote the set of actions that the processes can perform respectively. a and b stand for the atomic actions and c is the name of channel.

SKIP represents a process which does nothing but terminates successfully. *STOP* denotes that the process is in the state of deadlock and does nothing. The notation $a \rightarrow P$ represents that the process first engages in action a , then the subsequent behavior is like P . $P; Q$ performs P and Q sequentially. $c?x \rightarrow P$ gets a message through channel c and assigns it to a variable x , then behaves like P . $c!x \rightarrow P$ sends a message x using channel c , then behaves like P . The notation $a \rightarrow P \square b \rightarrow Q$ means that the behavior is like either P or Q and the selection is determined by the environment. $P \setminus S$ behaves like P except all the actions in set S are concealed. The notation $P \parallel Q$ describes the concurrent between P and Q . $P[X]Q$ represents that P and Q perform the concurrent events on set X of channels. $P[a \leftarrow b]$ indicates that a is replaced by b . The notation $P \triangleleft b \triangleright Q$ means if the condition b is true, the behavior is like P , otherwise, like Q . *CHAOS*(x) can perform any sequence of events from its alphabet x .

III. FORMALIZING THE ARIADNE PROTOCOL

In this section, we use CSP to model the route discovery of the Ariadne protocol. Firstly, we define the sets and channels that we would use below.

We assume the existence of the set **Initiator** of initiators, **Target** of targets, **Node** of intermediate nodes and **Intruder** of the internal nodes captured by the malicious nodes. The set **SharedKey** contains the keys shared between initiator and target and **Key** involves the TESLA keys that the intermediate nodes use. In addition, we also define four types of the messages as follows:

$$\begin{aligned}
 MSG1 &=_{df} \{ msg_{req}.h_s.S. * | h_s = MAC(K_{SD}, msg_{req}), \\
 &\quad K_{SD} \in SharedKey, S \in Initiator, \\
 &\quad * \in Node \cup Target \} \\
 MSG2 &=_{df} \{ msg_{req}.h_{X_i}.(..X_j..X_i).(..M_{X_j}..M_{X_i}).X_i. * | \\
 &\quad M_{X_i} = MAC(K_{X_{t_i}}, msg_{req}, h_{X_i}, \\
 &\quad \quad (..X_j..X_i).(..M_{X_j}..M_{X_{i-1}})), \\
 &\quad h_{X_i} = H(X_i, h_{X_{i-1}}), K_{X_{t_i}} \in Key, \\
 &\quad X_j, X_i \in Node, i \in N,
 \end{aligned}$$

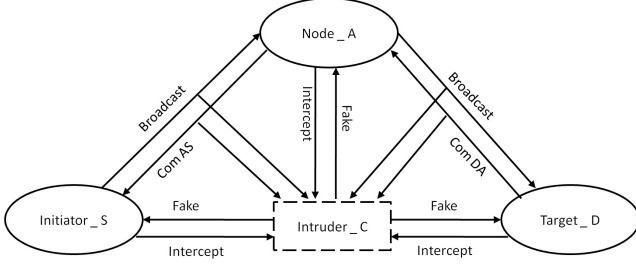


Figure 2. Communications between nodes using channels

$$\begin{aligned}
 & * \in \text{Node} \cup \text{Target} \} \\
 \text{MSG3} &=_{df} \{ \text{msg}_{rep}.M_D.D.X_i | \\
 & M_D = \text{MAC}(K_{DS}, \text{msg}_{rep}), \\
 & K_{DS} \in \text{SharedKey}, i \in N, \\
 & X_i \in \text{Node}, D \in \text{Target} \} \\
 \text{MSG4} &=_{df} \{ \text{msg}_{rep}.M_D.(..K_{X_{j_{t_i}}}..K_{X_{i_{t_i}}}).X_i.S | \\
 & M_D = \text{MAC}(K_{DS}, \text{msg}_{rep}), \\
 & K_{X_{j_{t_i}}}, K_{X_{i_{t_i}}} \in \text{Key}, i \in N, \\
 & X_j, X_i \in \text{Node}, S \in \text{Initiator} \} \\
 \text{MSG} &=_{df} \text{MSG1} \cup \text{MSG2} \cup \text{MSG3} \cup \text{MSG4}
 \end{aligned}$$

Here, *MSG1* represents the set of broadcast messages which are sent by the initiator to the intermediate nodes or the target. The set of *MSG2* stores the messages that are broadcasted by the intermediate nodes. The messages in the set of *MSG3* stand for the reply messages which are sent by the target to some intermediate node. And the set of *MSG4* holds the reply messages that are sent back to the initial node.

We use four channels to model the communication in the process of the Ariadne protocol: *Broadcast*, *Com_{X_iX_j}*, *Intercept*, *Fake*.

- *Broadcast*: it is used to broadcast and re-broadcast the message.
- *Com_{X_iX_j}*: it denotes the standard communication between nodes X_i and X_j .
- *Intercept*: the intruder uses it to intercept the information from normal communications.
- *Fake*: it is used by the intruder to send messages which are modified to the normal nodes.

In addition, we also define another two channels: *Session* and *Fake_session*, which represent a successful communication and a successful intrusion respectively.

The declaration of the channels are as follows:

Channel *Broadcast*, *Com_{X_jX_i}*, *Intercept*, *Fake*: MSG

Channel *Session*, *Fake_session*: Initiator.Target

Figure 2 illustrates the communications between nodes using channels.

A. Clock

The clock is the shared communication entity among the initiator, intermediate nodes and the target. Through clock synchronization and the delaying of releasing the key, the protocol which uses the TESLA broadcasting authentication mechanism achieves the desired effect of asymmetric key encryption.

$$\begin{aligned}
 \text{Clock}(i) &=_{df} (\text{tick} \rightarrow \text{Clock}(i+1)) \\
 &\quad \square (\text{time?request} \rightarrow \text{time!i} \rightarrow \text{Clock}(i))
 \end{aligned}$$

where $i \geq 0 \wedge i \in \mathbb{N}$.

Note that, if *Clock* receives a request from the channel *time*, it will return the current time *i*.

B. Initiator

In the whole process of route discovery, initiator will broadcast the route request and receive the route reply. Besides, it also has to check the validity of each key in the key list and the correctness of the MAC value of the target. Without considering the intruders, we model the behaviors of the initiator node as follows:

$$\begin{aligned}
 \text{Initiator}_1(S, T_0, t_{int}) &=_{df} \text{time!request} \rightarrow \text{time?T}_{S_0} \rightarrow \\
 &\quad \text{STOP} \triangleleft (T_{S_0} > (T_0 + (T+1) * t_{int}) | T_{S_0} < T_0) \triangleright \\
 \text{Initiator}_2(S, D, K_{SD}, K_{DS}) &
 \end{aligned}$$

The time beginning to send route request is T_{S_0} which must be within the time period from T_0 to T_{T+1} . The time period is divided into $T+1$ small time intervals with a length of t_{int} for each one.

$$\begin{aligned}
 \text{Initiator}_2(S, D, K_{SD}, K_{DS}) &=_{df} |||_{X_i \in \text{Node} \wedge i \in [1, n]} (\\
 &\quad \text{Broadcast!msg}_{req}. \text{MAC}(K_{SD}, \text{msg}_{req}).S.X_i \rightarrow \\
 &\quad \text{Com}_{X_i S}?\text{msg}_{rep}. \text{MAC}(K_{DS}, \text{msg}_{rep}).\overrightarrow{K_{t_i}}.X_i.S \rightarrow \\
 &\quad \text{STOP} \triangleleft (\text{MAC}(K_{SD}, \text{msg}_{rep}) \neq \\
 &\quad \text{MAC}(K_{DS}, \text{msg}_{rep})) \triangleright \text{CheckKeyValid}_1(\overrightarrow{NL}, \overrightarrow{K_{t_i}}) \\
 &\quad \rightarrow \text{Session}.S.D) \rightarrow \text{Initiator}_1(S, T_0, t_{int})
 \end{aligned}$$

$\overrightarrow{K_{t_i}}$ and $\overrightarrow{NL}$ stand for the key list and the node list respectively. The process of *CheckKeyValid₁* is used to check the validation of any key in the key list by the initiator and we give its definition as follows:

$$\begin{aligned}
 \text{CheckKeyValid}_1(\overrightarrow{NL}, \overrightarrow{K_{t_i}}) &=_{df} \\
 &\quad \text{SKIP} \triangleleft (\text{len}(\overrightarrow{K_{t_i}}) == 0) \triangleright \\
 &\quad (\text{CheckKeyValid}_1(\text{tail}(\overrightarrow{NL}), \text{tail}(\text{reverse}(\overrightarrow{K_{t_i}}))) \triangleleft \\
 &\quad (\text{keymap}(\text{head}(\overrightarrow{NL})) == F^{i-j}(\text{head}(\text{reverse}(\overrightarrow{K_{t_i}})))) \\
 &\quad \triangleright \text{STOP}))
 \end{aligned}$$

len shows the length of the list. *head* and *tail* return the first element and the reminder of a list respectively. *keymap* is a function to get the authenticated key at time t_j according to the node id and through $F^{t_i-t_j}(K_{t_i})$ one can get the key K_{t_j} at time t_j . Then, we apply this equation to the received key value to determine if the computed value matches a previous known authentic key value at time t_j on the key chain. Here, *reverse* is also a function to reverse

the elements in the key list $\overrightarrow{K_{t_i}}$.

Indeed, we must allow the possibility of intruder actions, such as intercepting the request and faking the reply. We model this situation via the renaming of CSP:

$$\begin{aligned} Initiator(S, T_0, t_{int}) &=_{df} Initiator_1(S, T_0, t_{int}) \\ &[[Broadcast!msg_{req}.MAC(K_{SD}, msg_{req}).S.X_i \\ &\leftarrow Broadcast!msg_{req}.MAC(K_{SD}, msg_{req}).S.X_i, \\ &Broadcast!msg_{req}.MAC(K_{SD}, msg_{req}).S.X_i \\ &\leftarrow Intercept!msg_{req}.MAC(K_{SD}, msg_{req}).S.X_i, \\ &ComX_iS?msg_{rep}.MAC(K_{DS}, msg_{rep}).\overrightarrow{K_{t_i}}.X_i.S \\ &\leftarrow ComX_iS?msg_{rep}.MAC(K_{DS}, msg_{rep}).\overrightarrow{K_{t_i}}.X_i.S, \\ &ComX_iS?msg_{rep}.MAC(K_{DS}, msg_{rep}).\overrightarrow{K_{t_i}}.X_i.S \\ &\leftarrow Fake?msg_{rep}.MAC(K_{DS}, msg_{rep}).\overrightarrow{K_{t_i}}.X_i.S, \\ &Session.S.D \leftarrow Session.S.D, \\ &Session.S.D \leftarrow Fake_session.S.D]] \end{aligned}$$

C. Node

Because the intermediate node uses the TESLA for route data authentication, after it receives the request, it checks whether the key the last node used has already been released or not. If the key is released, the node will discard the request. In the model below, X_l stands for the last node, X_i for the current node and X_n for the next node.

$$\begin{aligned} Node_1(X_l, X_i, K_{X_{i_{t_i}}}, \delta, T_{S_0}, t_{int}) &=_{df} \\ &(((X_n \in Node \wedge n \in N \wedge X_n \neq X_i) \rightarrow \\ &Broadcast?msg_{req}.h_{X_l}.NL_{X_l}.MAC_{X_l}.X_l.X_i \rightarrow \\ &time!request \rightarrow time?T_{S_1} \rightarrow \\ &CheckkeyValid_2(T_{S_0}, \delta, t_{int}) \rightarrow \\ &Broadcast!msg_{req}.h_{X_l}.NL_{X_l}.MAC_{X_l}.X_i.X_n \rightarrow \\ &ComX_nX_i?msg_{rep}.MAC(K_{DS}, msg_{rep}).\overrightarrow{K_{X_{n_{t_i}}}}.X_n.X_i \\ &\rightarrow time!request \rightarrow time?T_{S_2} \rightarrow \\ &WaitKeyReleased(T_{S_0}, \delta, t_{int}) \rightarrow \\ &ComX_iX_l!msg_{rep}.MAC(K_{DS}, msg_{rep}).\overrightarrow{K_{X_{i_{t_i}}}}.X_i.X_l) \\ &\rightarrow Node_1(X_l, X_i, K_{X_{i_{t_i}}}, \delta, T_{S_0}, t_{int})) \end{aligned}$$

$\overrightarrow{h_{X_i}}$ means the hash chain constructed using the per-hop hash function $H(x)$ and $\overrightarrow{MAC_{X_i}}$ stands for the current MAC list. $CheckKeyValid_2$ is a process to check the TESLA key the last node used is released or not and $WaitKeyReleased$ is another process used for node X_i itself to wait until its TESLA key is released at some time. Here, δ represents the number of the time intervals from starting using the key to releasing it. We can also use renaming to model the behaviors of the node so as to consider the actions of the intruder as process of $Node$, and here we omit the detailed model.

D. Target

After the target receives the route request, it checks whether there is any key the nodes use has been released or not through the process $CheckKeyValid_3$ and it also checks the MAC value of the route request using K_{DS} . If there is no error, it will unicast a route reply according to

the reverse order of the nodes in the node list. Here, the key list $\overrightarrow{K_{t_i}}$ is an empty list and $\overrightarrow{\Delta t}$ is a list which holds the time difference between every two nodes.

$$\begin{aligned} Target_0(X_l, D, K_{DS}, \delta, t_{int}, T_{S_0}, \overrightarrow{\Delta t}) &=_{df} \\ &Broadcast?msg_{req}.h_{X_l}.NL_{X_l}.MAC_{X_l}.X_l.D \rightarrow \\ &time!request \rightarrow time?T_{S_4} \rightarrow \\ &CheckKeyValid_3(NL_{X_l}, \delta, t_{int}, T_{S_0}, \overrightarrow{\Delta t}) \rightarrow \\ &ComDX_l!msg_{rep}.MAC(K_{DS}, msg_{rep}).\overrightarrow{K_{t_i}}.D.X_l \rightarrow \\ &Session.S.D \rightarrow Target_0(X_l, D, K_{DS}, \delta, t_{int}, T_{S_0}, \overrightarrow{\Delta t}) \end{aligned}$$

In addition, with respect to the intruder, it can intercept the reply message or fake the request message. We model the target via renaming as follows:

$$\begin{aligned} Target(X_l, D, K_{DS}, \delta, t_{int}, T_{S_0}, \overrightarrow{\Delta t}) &=_{df} \\ &Target_0(X_l, D, K_{DS}, \delta, t_{int}, T_{S_0}, \overrightarrow{\Delta t}) \\ &[[Broadcast?msg_{req}.h_{X_l}.NL_{X_l}.MAC_{X_l}.X_l.D \\ &\leftarrow Broadcast?msg_{req}.h_{X_l}.NL_{X_l}.MAC_{X_l}.X_l.D, \\ &Broadcast?msg_{req}.h_{X_l}.NL_{X_l}.MAC_{X_l}.X_l.D \\ &\leftarrow Fake?msg_{req}.h_{X_l}.NL_{X_l}.MAC_{X_l}.X_l.D, \\ &ComDX_l!msg_{rep}.MAC(K_{DS}, msg_{rep}).\overrightarrow{K_{t_i}}.D.X_l \\ &\leftarrow ComDX_l!msg_{rep}.MAC(K_{DS}, msg_{rep}).\overrightarrow{K_{t_i}}.D.X_l, \\ &ComDX_l!msg_{rep}.MAC(K_{DS}, msg_{rep}).\overrightarrow{K_{t_i}}.D.X_l \\ &\leftarrow Intercept!msg_{rep}.MAC(K_{DS}, msg_{rep}).\overrightarrow{K_{t_i}}.D.X_l, \\ &Session.S.D \leftarrow Session.S.D, \\ &Session.S.D \leftarrow Fake_session.S.D]] \end{aligned}$$

E. Intruder

In this paper, we also propose an intruder model. Here, the intruder is regarded as a process which can intercept, eavesdrop and fake the message. Through learning, the intruder can get much information about the route discovery. We define a set $Fact$ explaining all the facts it learns.

$$\begin{aligned} Fact &=_{df} Initiator \cup Node \cup Target \cup Key \\ &\cup SharedKey \cup MSG \cup \{F(K)\} \\ &\cup \{MAC(K, msg)\} \cup \{H(node, H')\} \end{aligned}$$

Besides, the intruder node can also deduce some facts from what it has known. We denote $I \mapsto f$ to represent that the intruder deduces new fact f from the known set I .

- 1) $\{F(K_{X_i})\} \mapsto K_{X_{i-1}}, K_{X_{i-2}}, \dots, K_{X_0}$
- 2) $\{MAC(K, msg)\} \mapsto MAC(K, msg)$
- 3) $\{H(node, H')\} \mapsto H(node, H')$
- 4) $I \mapsto f \wedge I \subseteq \mathbf{I} \Rightarrow \mathbf{I} \mapsto f$

As stated in [6], we also assume that the intruder knows the identity of each node so that it can get some information from the fact without deducing. For example:

$$\begin{aligned} info(msg.h_S.S.*) &= \{msg, h_S, S, *\} \\ info(msg.h_{X_i}.MAC.X_i.*) &= \{msg, h_{X_i}, MAC, X_i, *\} \end{aligned}$$

In order to model the behaviors of the intruder, we define another channel *deduce*, through which the intruder can deduce some new facts from the set of the facts it knew. The declaration of channel *deduce* is:

Channel *deduce* : $fact.\{Fact\}$.

The model of the intruder is as follows:

```

Intruder0 =df
□m ∈ MSG Broadcast.m → Intruder0(I ∪ info(m))
□
□m ∈ MSG ComXiXj.m → Intruder0(I ∪ info(m))
□
□m ∈ MSG Intercept.m → Intruder0(I ∪ info(m))
□
□m ∈ MSG Fake.m → Intruder0(I)
□
□f ∈ Fact, f ∉ I, I → f deduce.f.I → Intruder0(I ∪ f)

```

We hide the *deduce* channel because of the internal actions, then we get the model of the intruder:

$Intruder =_{df} Intruder_0 \setminus \{\{deduce\}\}$

F. System and Specification

The whole system can be modeled as the parallel composition of the initiator, the intermediate nodes and the target. All these communication entities share the same clock. First we consider the system without intruder.

```

INITIATOR =df Clock(0)[[time]]Initiator(S, T0, tint)
NODE =df Clock(0)[[time]]
      Node(XI, Xi, KXiti, δ, TS0, tint)
TARGET =df Clock(0)[[time]]
      Target(XI, D, KDS, δ, tint, TS0, Δt)
SYSTEM0 =df INITIATOR
      [[Broadcast, ComXiS, Session]]NODE
      [[Broadcast, ComDXi, Session]]TARGET

```

Then, we add the intruder into the whole system.

```

SYSTEM =df SYSTEM0[[INTRUDER_ALPH]]
      INTRUDER(S, D, C, KCi)
INTRUDER_ALPH =df {Broadcast, ComXiXj,
      Intercept, Fake, Session, Fake_session}.

```

Ariadne aims to preventing a malicious node from compromising the route. The initiator or the target can not accept any message modified by the intruder. Here, we define the specification for the security property of the Ariadne protocol as:

$SPEC =_{df} CHAOS(\sum -|Fake_session.S.D|)$.

Note that $\sum$ stands for the set of all the events. Some specific security properties will be listed in the next section.

IV. VERIFICATION IN PAT

In this section, we use PAT to implement our CSP model of the Ariadne protocol. First we give a brief introduction to PAT. PAT (Process Analysis Toolkit) is designed as an extensible and modularized framework based on CSP and it has been used to model and verify a lot of different systems for varieties of properties such as deadlock-freeness, divergence-freeness, reachability, LTL properties with fairness assumptions, refinement checking and probabilistic model checking. We list some notations in PAT as follows:

- 1) *#define N 0* defines a global constant *N* which has the initial value 0.
- 2) *var msglist[N]* defines an array named *msglist* and the size of it is *N*.
- 3) *Channel c 5* defines a communication channel and the capacity of it is 5.
- 4) $P = \{x = x + 1\} \rightarrow Skip$ defines an event that can be attached with assignment using which we can update the value of a global variable *x*.
- 5) $c!a.b \rightarrow P$ and $c?x.y \rightarrow P$ refer to sending message *a.b* and receiving message from channel *c* respectively.

Besides, PAT can also describe *if-then-else* and *while*, etc. More details about this tool can be found in [18].

A. The Ariadne Protocol in PAT

Here, we implement three cases of our model using PAT. We define four other functions in a new *C#* library as follows:

- *MACValue* is used only for the initiator and the target to compute their MAC values using the shared key and the message.
- *HashValue* is used to compute the hash values for the intermediate nodes through function *H(x)*.
- *mediaMACValue* is used for the intermediate nodes to compute the MAC values using their corresponding TESLA keys.
- *KeyValue* is a function to compute the node's TESLA key at some time on the TESLA chain.

We also need some significant channels and variables, e.g. *broadcast* stands for the broadcast channel, *N* represents the number of the intermediate nodes in our case, *msgreq* stands for the request message, *idlist[N + 2]* is a list that stores the identity for each node, *msgreply[N + 1][2]* is a two-dimensional array that stores the reply messages, *distancelist* is also a list records the distance between each two nodes. We give the declarations of them as follows:

```

#define N 4;          channel broadcast (N + 1) * N;
#define msgreq 10;    var msglist[N + 1][4];
var idlist[N + 2] = [0, 1, 2, 3];    var msgreply[N + 1][2];
var distancelist[N + 2][N + 2] =
    [0, 1, 2, 3, -1, 0, 1, 2, -1, -1, 0, 1, -1, -1, -1, 0];

```

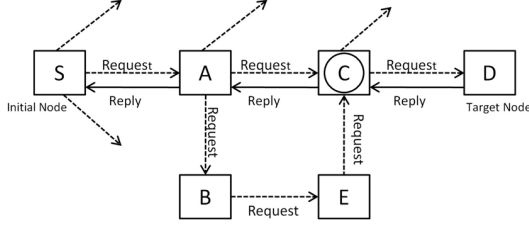


Figure 3. Case II with Active-1-1 Intruder

Case I: We implement the basic process of route discovery without intruder. We assume that there are only four nodes such as S , A , B and D which have the same topology as in Figure 1.

Here, we give relevant code in PAT to show how the initiator node sends and receives messages. The initiator S broadcasts the request message to all of its neighbors and it also receives the reply message and does some appropriate checks such as checking the time validation which is interpreted by the process $CheckInitTime(clock)$. We give the PAT code of the processes $SendInits$, $SendInit$ and $SendInit2$ as follows:

```

SendInits(sender, content) = |||receiver : {1..N + 1}@
    (SendInit(sender, content, receiver));
SendInit(sender, content, receiver) =
    ifa((sender == receiver)||
        (idlist[sender] > idlist[receiver])||
        distancelist[sender][receiver] != 1){Skip}
    else
        {time!true -> time?x -> clock = x ->
            CheckInitTime(clock);
            SendInit2(sender, content, receiver)};
SendInit2(sender, content, receiver) =
    {MACInit = call(MACValue, KeySD, msgreq)}
    -> broadcast!sender.receiver.content.MACInit
    -> {msglist[sender][0] = sender;
        msglist[sender][1] = content;
        msglist[sender][2] = MACInit;
        msglist[sender][3] = 0} -> Skip;

```

In the code section, some parameters from the model are defined as the global variables which we have already mentioned before. Here, the process of the whole system is represented:

```

System() = Clock(0)|||(Initiator(1)
    |||Nodes())|||Target(3));

```

Case II: Considering the Active-1-1 intruder, the intruder owns only one compromised node. We add the process of the intruder into the system and the topology is shown in Figure 3, in which we use a circle to identify the intruder node. In the code, the id corresponding to the node (S, A, B, E, C, D) is $(0, 1, 2, 3, 4, 5)$.

When the intruder C receives the message $\langle msg_{req}, h_A, (A), (M_A) \rangle$ from A , it will wait until the message $\langle msg_{req}, h_E, (A, B, E), (M_A, M_B, M_E) \rangle$ from E arrives. Then C will change the message especially the node list as (A, B, C) to D . The modified message is $\langle msg_{req}, h_C, (A, B, C), (M_A, M_B, M_C) \rangle$, where $h_C = H(C, H(B, h_A))$ and M_B can be deduced from the known information. After C gets the reply from D , it will cache the reply message until its own TESLA key and the TESLA key of node B have been released, then it will send the reply to A directly. In fact, (S, A, B, C) is a fake path that S does not recognize. This attack has been mentioned in [3], here, we implement it using PAT. We give relevant code about the actions of the intruder in Case II:

```

Intruder(i) = broadcast?sender.i.msg.mac ->
    save.i{intruderlist[j][0] = sender;
        intruderlist[j][1] = i; intruderlist[j][2] = msg;
        intruderlist[j][3] = mac; j = j + 1;
        cnt_intruder = cnt_intruder + 1} ->
    Change(i, j);
    ifa(cnt_intruder >= 2){Skip}
    else{Intruder(i)};
Change(i, k) = ifa(k == 2)
    {change.i{msglist[i - 1][0] = i;
        msglist[i - 1][2] =
            call(HashValue, i, msglist[i - 2][2]);
        msglist[i - 1][3] =
            call(mediaMACValue, KeyTi[i - 1],
                msglist[i - 1][1], msglist[i - 1][2],
                i, msglist[i - 2][3])} ->
        Sendmedias(i, msglist[i - 1][1], msglist[i - 1][2]);
        ReceiveReply(i)} else{Skip};

```

Case III: Here, we consider the Active-1-2 intruder, which means the intruder controls two corrupted nodes but they use the same compromised identifier. The topology is shown in Figure 4 and we use a circle to identify the intruder nodes. The id corresponding to the node (S, A, B, C, D) is $(0, 1, 2, 3, 4)$. When C receives the request from A , it saves the message into his private list, which the other corrupted node can also access. Then, the intruder whose identifier is also C will receive the message from B , it will change the node list as (A, C) to D . After C receives the reply message from D , it will also put it into its own list, then the other node whose identity is also named C will send the message back to A ignoring the intermediate node B . Relevant code is represented below:

```

Intruder(i) = broadcast?sender.i.msg.mac ->
    save.i{intruderlist[j + 1][0] = sender;
        intruderlist[j + 1][1] = i; intruderlist[j + 1][2] = msg;
        intruderlist[j + 1][3] = mac} -> Change(i);
Change(i) = change.i{msglist[i - 1][0] = i;
    msglist[i - 1][2] =
        call(HashValue, i, msglist[i - 1][1]);
    msglist[i - 1][3] =
        call(mediaMACValue, KeyTi[i - 1],

```

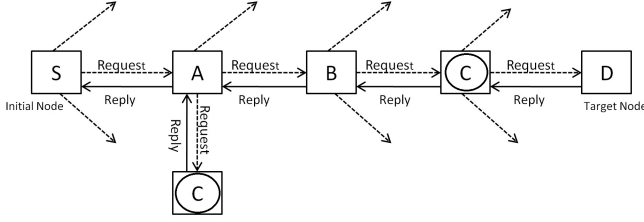


Figure 4. Case III with Active-1-2 Intruder

$msglist[i-1][1], msglist[i-1][2],$
 $i, intruderlist[0][3]\} - >$
Sendmedias($i, msglist[i-1][1], msglist[i-1][2]$);
ReceiveReply(i);

B. Verification

Based on the trace analysis, we define the assertions of some properties in Table 3 and we list four properties as follows:

Property 1: Deadlock Freedom

$\forall i \in (Initiator \cup Node \cup Target) \bullet$
 $(clock \leq TimeMax \wedge i.send == true) \Rightarrow$
 $(i.receive == true \wedge num(send) == num(receive))$

Within the specific time, if there is some node sending message through the channel, there must be some node waiting to receive the message and the number of the sent message is equal to the number of the received message.

Property 2: Message Consistency

$\exists m, n \in \mathbb{N} \bullet (((request == msglist[m][1]) \Rightarrow$
 $(\forall j request == msglist[j][1])) \wedge$
 $((reply == replylist[n][0]) \Rightarrow$
 $(\forall k reply == reply[k][0])))$

All the request messages, especially the message received by the target node must be consistent with the request sent by the initiator node. And the reply message received by all the nodes, including the initiator node, is the same with the message sent by the target node.

Property 3: Node List Security

$\forall i, j \in NID \bullet (nodelist[i] == requestlist[i][0]) \Rightarrow$
 $(replylist[j+1][2] == nodelist[j])$
 where $NID =_{df} 0..N$

The Ariadne protocol aims to prevent a malicious node from compromising the route. It wants to ensure the security of the node list that no intruder can change the order of the nodes or add or remove any node from the node list. So the initiator receives the node list, which must be the same with that the target node receives.

Property 4: Fake Path Existence

$\exists k \in NID \bullet (nodelist[k] == msglist[k][0]) \Rightarrow$
 $(distance[msglist[k][0]][msglist[k+1][0]] = 1)$
 where $NID =_{df} 0..(N-1)$

Table 3. Assertions of properties

```
#define goal1 ((msgreq==msgReced)&&
(msgrep==msgRepReced));
#assert System() reaches goal1;

#define goal2 ((msglist[1][0]==1)&&(msglist[2][0]==2)&&
(msglist[3][0]==4));
#assert System() reaches goal2;

#define goal3((call(KeyValue,msgreply[1][1])!=KeyTj[1]||
(call(KeyValue,msgreply[0][1])!=KeyTj[0])||
(call(MACValue,KeySD,msgreply[1][0])!=MACTar);
#assert System() reaches goal3;

#define goal4 ((distancelist[msglist[1][0]][0]==1)&&
(distancelist[msglist[2][0]][msglist[1][0]]==1)&&
(distancelist[msglist[3][0]][msglist[2][0]]==1));
#assert System() reaches goal4;
```

C. Results analysis

The result of the verification is shown in Table 4. Here, P is the abbreviation of Pass which means the case study reaches the goal and pass the verification of the property. And F, shorted for Fail, means the case does not reach the goal.

Through the table, we see that Case I reaches three goals and does not meet the last one because of no fake path existing in Case I. Conversely, both Case II and Case III pass the verification of property 4 indicating there are fake paths in these two case studies. We also give the User Interface Figure, Figure 5, which shows the result of the verification of Case II in PAT.

Table 4. The result of the verification

	Property 1	Property 2	Property 3	Property 4
Case Study 1	P	P	P	F
Case Study 2	P	P	P	P
Case Study 3	P	P	P	P

V. CONCLUSION AND FUTURE WORK

In this paper, we proposed a CSP model of the Ariadne secure route protocol. All the communication entities of the protocol in our model, involving the initiator node, the intermediate nodes, and the target, have been abstracted as processes respectively. They share the same clock to realize the effect of asymmetric key encryption through clock synchronization and detention. Besides, we also proposed an intruder model in which the intruder can perform the attacks. Furthermore, we use PAT, a model checker to implement and validate our formal model automatically. Our verification result demonstrates that some fake paths indeed exist in the Ariadne protocol.

In the future, we would work on the formalizing of the other phase of the Ariadne protocol, including the process

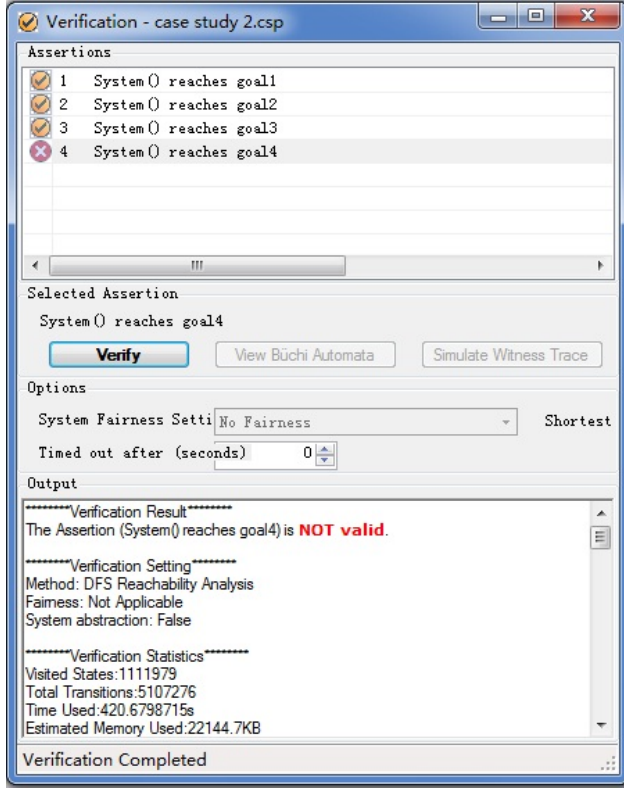


Figure 5. The result of the verification of Case II

of the route maintain. Also, based on the whole achieved model of the Ariadne protocol, further verification by using PAT is also an interesting topic to be explored.

Acknowledgement The authors gratefully acknowledge support from the Danish National Research Foundation and the National Natural Science Foundation of China (Grant No. 61061130541) for the Danish-Chinese Center for Cyber Physical Systems. This work was also supported by National Basic Research Program of China (No. 2011CB302904), National High Technology Research and Development Program of China (No. 2011AA010101), National Natural Science Foundation of China (No. 61021004), and Shanghai Leading Academic Discipline Project (No. B412).

REFERENCES

- [1] G. Ács, L. Buttyán, and I. Vajda. Provably Secure On-Demand Source Routing in Mobile Ad Hoc Networks. *IEEE Trans. Mob. Comput.*, 5(11):1533–1546, 2006.
- [2] S. D. Brookes, C. A. R. Hoare, and A. W. Roscoe. A Theory of Communicating Sequential Processes. *J. ACM*, 31(3):560–599, 1984.
- [3] L. Buttyán and I. Vajda. Towards provable security for ad hoc routing protocols. In *Proc. 2nd ACM workshop on Security of ad hoc and sensor networks*, SASN '04, pages 94–105, New York, NY, USA, 2004. ACM.
- [4] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice-Hall, 1985.
- [5] Y.-C. Hu, D. B. Johnson, and A. Perrig. SEAD: secure efficient distance vector routing for mobile wireless ad hoc networks. *Ad Hoc Networks*, 1(1):175–192, 2003.
- [6] Y.-C. Hu, A. Perrig, and D. B. Johnson. Ariadne: A Secure On-Demand Routing Protocol for Ad Hoc Networks. *Wireless Networks*, 11(1-2):21–38, 2005.
- [7] D. B. Johnson and D. A. Maltz. Dynamic Source Routing in Ad Hoc Wireless Networks. *Mobile Computing*, pages 153–181, 1996.
- [8] C.-H. Lin, W.-S. Lai, Y.-L. Huang, and M.-C. Chou. Secure Routing Protocol with Malicious Nodes Detection for Ad Hoc Networks. *Advanced Information Networking and Applications Workshops, International Conference on*, 0:1272–1277, 2008.
- [9] G. Lowe and J. Davies. Using CSP to Verify Sequential Consistency. *Distributed Computing*, 12(2-3):91–103, 1999.
- [10] G. Lowe and A. W. Roscoe. Using CSP to Detect Errors in the TMN Protocol. *IEEE Trans. Software Eng.*, 23(10):659–669, 1997.
- [11] T. Mazur and G. Lowe. Counter Abstraction in the CSP/FDR setting. *Electr. Notes Theor. Comput. Sci.*, 250(1):171–186, 2009.
- [12] A. Perrig, R. Canetti, D. X. Song, and J. D. Tygar. Efficient and Secure Source Authentication for Multicast. In *Proc. the Network and Distributed System Security Symposium*. The Internet Society, 2001.
- [13] A. Perrig, R. Canetti, J. D. Tygar, and D. X. Song. Efficient Authentication and Signing of Multicast Streams over Lossy Channels. In *IEEE Symposium on Security and Privacy*, pages 56–73, 2000.
- [14] M.-L. Pura, I. Bica, and V. V. Patriciu. On modeling and formally verifying secure explicit on-demand ad hoc routing protocols. In *Proc. 2nd International Conference on Software Technology and Engineering*, volume 2, pages 215–220, 10 2010.
- [15] G. T. Rohrmair and G. Lowe. Using CSP to Detect Insertion and Evasion Possibilities within the Intrusion Detection Area. In *Proc. 1st International Conference on Formal Aspects of Security*, pages 205–220. Springer, 2002.
- [16] K. Sanzgiri, B. Dahill, B. N. Levine, C. Shields, and E. M. Belding-Royer. A Secure Routing Protocol for Ad Hoc Networks. In *Proc. 10th IEEE International Conference on Network Protocols*, pages 78–89. IEEE Computer Society, 2002.
- [17] K. A. Sivakumar and M. Ramkumar. Improving the resiliency of Ariadne. In *Proc. 9th IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks*, pages 1–6. IEEE, June 2008.
- [18] J. Sun, Y. Liu, and J. S. Dong. Model Checking CSP Revisited: Introducing a Process Analysis Toolkit. In *Proc. 3rd International Symposium on Leveraging Applications of Formal Methods, Verification and Validation*, pages 307–322. Springer, 2008.
- [19] M. G. Zapata and N. Asokan. Securing ad hoc routing protocols. In *Proc. 2002 ACM Workshop on Wireless Security*, pages 1–10. ACM, 2002.